FDAIE | Module 2 | Session 1

Data Pipeline Basics

ETL · ELT · Storage Layers · Full vs Incremental Load

Trainer	Programme
Umashankar K H	**Foundations of Data & AI Engineering**

Topics covered in today's session:

1. The Business Problem — Company X scenario
2. Where data comes from (source systems)
3. Storage layers: Raw Staging Warehouse
4. ETL — Extract, Transform, Load
5. ELT — Extract, Load, Transform
6. ETL vs ELT — when to use which
7. Full Load vs Incremental Load
8. Hands-on: DuckDB pipeline demo

1 The Business Problem

Everything we cover today is built around one real-world scenario:

STORY

You are a Data Engineer at Company X.

Stakeholders log in every morning at 9 AM expecting fresh, clean sales data.

Your deadline: data must be ready in the warehouse by 6 AM — every single day.

Data arrives overnight from three different source systems. Your job: collect it, clean it, make it trustworthy.

This is what a real data pipeline looks like. Today you learn how to build one.

2 Where Does the Data Come From?

In most companies, data does not live in one place. For Company X:

POS System / E-commerce platform → every order placed by a customer

CRM (Customer Relationship Mgmt) → customer profiles, names, regions

ERP / Inventory System → product catalogue, stock levels, categories

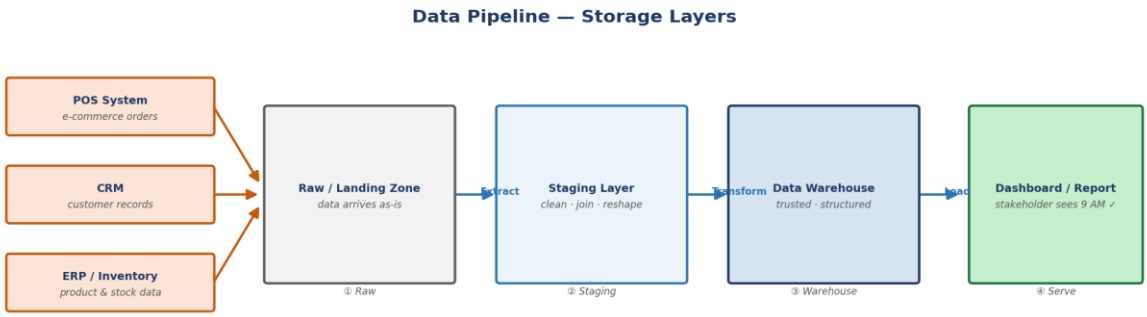
KEY POINT: These systems were NOT designed to talk to each other.

Each stores data in its own format, structure, and sometimes timezone.

The data engineer brings it all together — reliably, every day.

3 Storage Layers — Where Data Lives

Every pipeline moves data through distinct layers. Understanding these layers is fundamental.



Layer 1 — Raw / Landing Zone

- Data arrives here first, exactly as it came from the source. No changes, no cleaning.
- Contains nulls, duplicates, inconsistent formats — all of it.
- Think of it as your inbox — everything lands here, even the junk.

Layer 2 — Staging Layer

- The workbench. Data is cleaned, validated, joined, and reshaped here.
- Nothing here is final — it is a transition space between raw and warehouse.

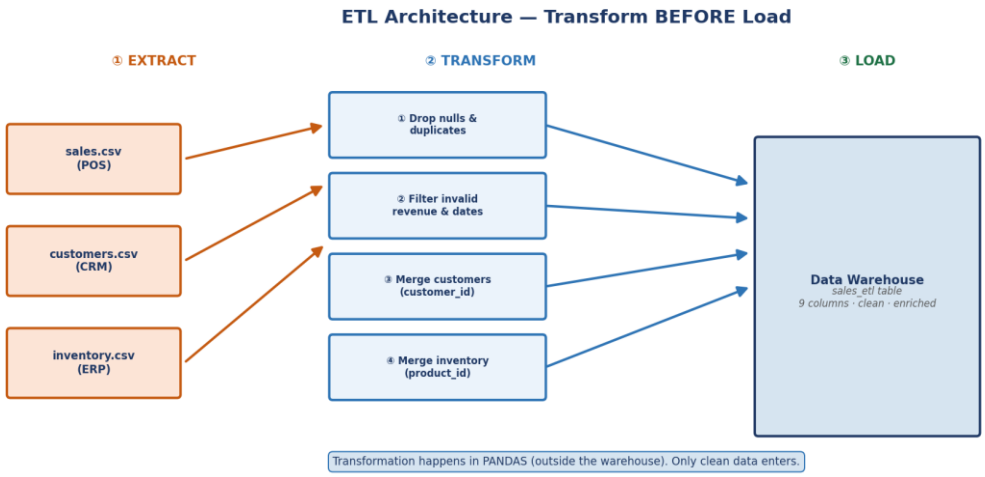
Layer 3 — Data Warehouse

- What the stakeholder sees at 9 AM. Clean, trusted, structured data.
- Dashboards and reports connect here. In production: Snowflake, BigQuery, Redshift.

4 ETL — Extract, Transform, Load

DEFINITION	Pull data from sources → Clean & join it (outside warehouse) → Load only clean data in

Core principle: transformation happens BEFORE data enters the warehouse.



Company X — ETL Step by Step

Step 1 Extract

- sales.csv — order_id, customer_id, product_id, order_date, revenue
- customers.csv — customer_id, name, region
- inventory.csv — product_id, product_name, category

Step 2 Transform (in Pandas — outside the warehouse)

- Drop rows with missing customer_id
- Remove duplicate order_ids
- Filter out negative revenue and future-dated orders
- Merge customers data → each order gets customer_name and region
- Merge inventory data → each order gets product_name and category

Step 3 Load

- Write enriched, clean table to the warehouse as sales_etl
- Result: 9-column table — order_id, order_date, revenue, customer_id, customer_name, region, product_id, product_name, category

ETL Code — Part 1 of etl_elt.py

```
# EXTRACT: read all 3 source files
sales_raw     = pd.read_csv(SALES)       # order_id, customer_id, product_id,
order_date, revenue
customers_raw = pd.read_csv(CUSTOMERS)   # customer_id, name, region
inventory_raw = pd.read_csv(INVENTORY)   # product_id, product_name, category

# TRANSFORM: clean + enrich in pandas (BEFORE loading)
sales_clean = sales_raw.replace("", pd.NA).dropna(subset=["customer_id"])
sales_clean = sales_clean.drop_duplicates(subset=["order_id"])
sales_clean = sales_clean[sales_clean["revenue"] >= 0]
sales_clean = sales_clean[pd.to_datetime(sales_clean["order_date"]) <=
pd.Timestamp.now()]

# Enrich: join customer name + region
sales_clean = sales_clean.merge(customers_raw, on="customer_id", how="left")
# Enrich: join product name + category
sales_clean = sales_clean.merge(inventory_raw, on="product_id",  how="left")

# LOAD: write enriched clean data to warehouse
con = duckdb.connect("warehouse.duckdb")
con.execute("CREATE OR REPLACE TABLE sales_etl AS SELECT * FROM sales_clean")
```

WHEN TO USE ETL

- Regulated industries (banking, healthcare) — must audit what entered the warehouse
- Structured, stable schemas
- Smaller data volumes where pre-load transform is feasible

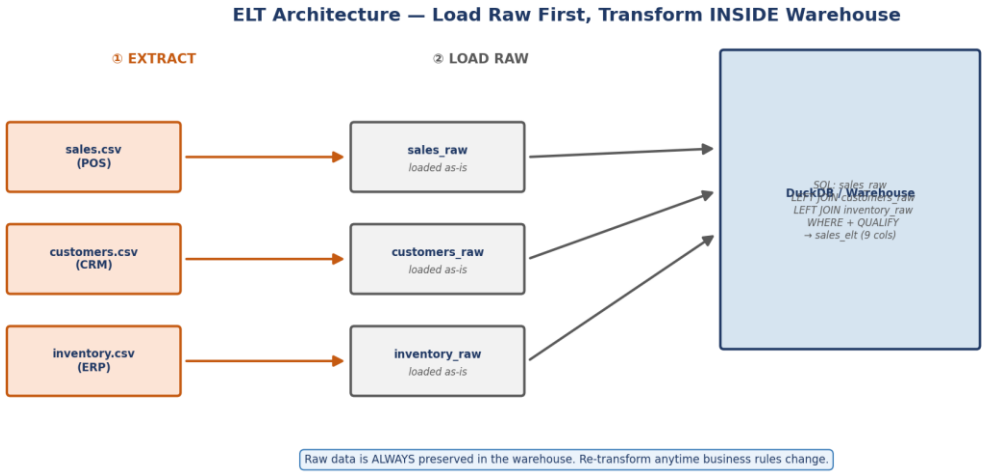
PRO: Warehouse stays clean — only trusted data stored.

CON: If business logic changes, you may need to re-pull raw data from the source.

5 ELT — Extract, Load, Transform

DEFINITION	Pull data from sources → Load raw data into warehouse → Transform using SQL inside

Core principle: raw data lands in the warehouse first. Transformation happens inside, in SQL.



Company X — ELT Step by Step

Step 1 Extract

- Same as ETL — pull from POS, CRM, and ERP

Step 2 Load Raw (no cleaning yet)

- All 3 tables land in DuckDB exactly as-is: sales_raw, customers_raw, inventory_raw
- Nulls, duplicates, messy formats — all present at this point

Step 3 Transform Inside the Warehouse (SQL)

- A single SQL query JOINs all 3 raw tables and applies filters
- QUALIFY removes duplicates using window functions — no pandas needed
- Output: sales_elt — same clean 9-column result, built entirely in SQL
- Raw tables remain intact — re-run the transform anytime

ELT Code — Part 2 of etl_elt.py

```
# LOAD: dump all 3 raw tables into DuckDB as-is (no cleaning yet)
con.execute(f"CREATE OR REPLACE TABLE sales_raw     AS SELECT * FROM
read_csv_auto('{SALES}')")
con.execute(f"CREATE OR REPLACE TABLE customers_raw AS SELECT * FROM
read_csv_auto('{CUSTOMERS}')")
con.execute(f"CREATE OR REPLACE TABLE inventory_raw AS SELECT * FROM
read_csv_auto('{INVENTORY}')")

# TRANSFORM: clean + join all 3 tables INSIDE the warehouse using SQL
con.execute("""
    CREATE OR REPLACE TABLE sales_elt AS
    SELECT
        s.order_id,
        s.order_date::DATE   AS order_date,
        s.revenue,
        c.customer_id,
        c.name               AS customer_name,
        c.region,
        i.product_id,
        i.product_name,
        i.category
    FROM sales_raw s
    LEFT JOIN customers_raw c ON s.customer_id = c.customer_id
    LEFT JOIN inventory_raw i ON s.product_id  = i.product_id
    WHERE s.customer_id IS NOT NULL AND s.customer_id != ''
      AND s.revenue >= 0
      AND s.order_date <= CURRENT_DATE
    QUALIFY ROW_NUMBER() OVER (PARTITION BY s.order_id ORDER BY s.order_date)
= 1
""")
```

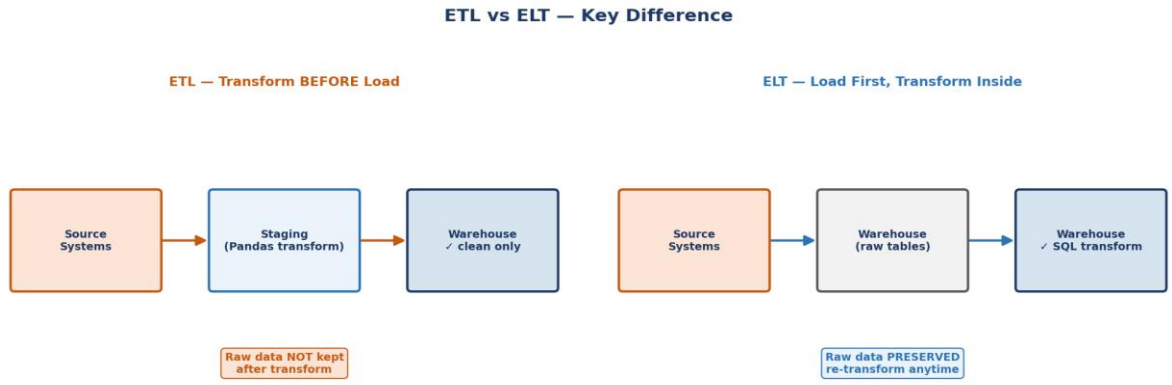
WHEN TO USE ELT

- Modern cloud warehouses (Snowflake, BigQuery) powerful enough to transform at scale
- Large / semi-structured data — JSON logs, clickstream, IoT events
- Business rules change often — raw data preserved, just rewrite the SQL
- Data science teams also need raw data access

PRO: Raw data always preserved — reprocess anytime.

CON: Storage costs are higher. Governance must be strong.

6 ETL vs ELT — Side by Side



Dimension	ETL	ELT
Transform happens	Before loading (in pandas)	After loading (SQL inside warehouse)
Raw data preserved?	No — only clean data stored	Yes — raw tables always available
Best for	Structured, regulated data	Large-scale, cloud-native pipelines
Tools	Pandas, Informatica, Talend	dbt, Fivetran, Airbyte + Snowflake
Speed to warehouse	Slower (clean first)	Faster (dump first)
Storage cost	Lower (clean data only)	Higher (raw + clean)
Rule changes	Re-pull raw data from source	Just rewrite the SQL transform

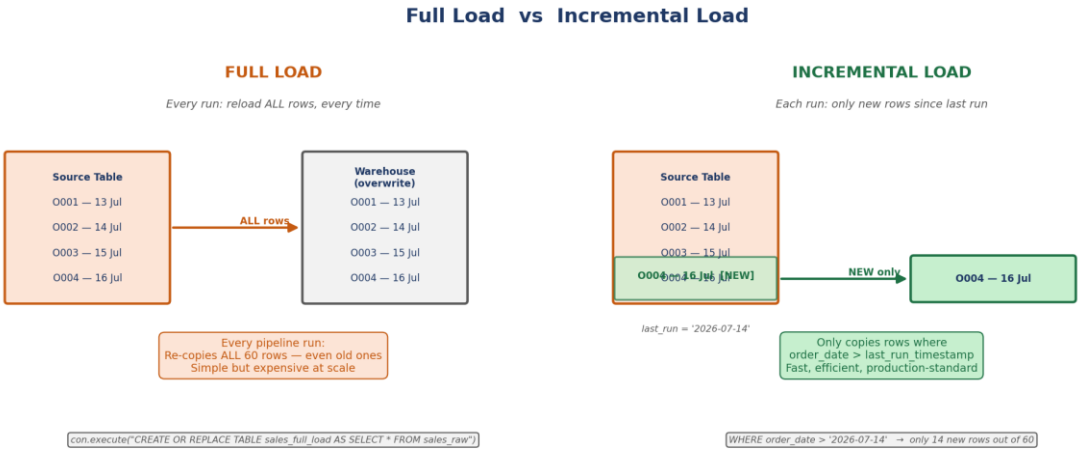
How to Choose

Ask yourself:

1. Must I audit exactly what entered the warehouse? ETL
2. Do I need fast ingestion of large data volumes? ELT
3. Will business rules change? Must I keep raw data? ELT
4. Is my data structured with stable schemas? ETL
5. Do data science teams also need raw access? ELT

7 Full Load vs Incremental Load

Once you know WHERE data goes (ETL or ELT), you need to decide HOW MUCH to move each run.



Dimension	Full Load	Incremental Load
What loads	ALL rows every run	Only NEW rows since last run
Speed	Slower (all data)	Faster (delta only)
Cost	Higher (more compute)	Lower (less compute)
Complexity	Simple	Needs a watermark column
When to use	Small datasets / first run	Production, daily pipelines
Risk	None from missing data	Missing rows if watermark fails

Full Load — Code from etl_elt.py

```
# Full load: reload ALL rows every pipeline run
con.execute("CREATE OR REPLACE TABLE sales_full_load AS SELECT * FROM sales_raw")

full_count = con.execute("SELECT COUNT(*) FROM sales_full_load").fetchone()[0]
print(f"Full load: {full_count} rows reloaded (every run, regardless of what changed)")
# Output: Full load: 60 rows reloaded (every run, regardless of what changed)
```

Incremental Load — Code from etl_elt.py

```
# Incremental load: only rows newer than the last successful run
last_run_timestamp = "2026-07-14"   # stored after each successful run

con.execute(f"""
    CREATE OR REPLACE TABLE sales_incremental AS
    SELECT * FROM sales_raw
    WHERE order_date > '{last_run_timestamp}'
""")

incr_count = con.execute("SELECT COUNT(*) FROM
sales_incremental").fetchone()[0]
print(f"Incremental load: only {incr_count} new rows since
{last_run_timestamp}")
# Output: Incremental load: only 14 new rows since 2026-07-14
```

What is a Watermark?

WATERMARK	A column (usually order_date or updated_at) that tells the pipeline where it last left off.

- Each run reads the last watermark timestamp from a config or metadata table
- Pipeline loads only rows newer than that timestamp
- After a successful run, the watermark is updated to the current timestamp
- Risk: if the source system does not update the watermark column correctly, rows can be missed

Company X uses order_date as the watermark.

"Give me all orders where order_date > 2026-07-14" = only yesterday's new orders.

This is far more efficient than reloading all 60+ rows every single night.

8 What is DuckDB?

DUCKDB	An in-process SQL analytics engine — like SQLite, but built for data analytics. No server needed.

- Zero setup — runs inside your Python script, no Docker or cloud account required
- Reads CSV, JSON, and Parquet files directly using read_csv_auto()
- Full SQL support — window functions, CTEs, JOINs, QUALIFY
- Same concepts you will use in Snowflake, BigQuery, or Redshift in production

In production, DuckDB is replaced by:

Snowflake / BigQuery / Redshift for cloud-scale data warehouses

dbt for SQL transformations inside those warehouses

Every SQL query you wrote today in DuckDB translates directly to those tools.

9 Key Takeaways

1	Data pipelines move data from source systems into a warehouse where stakeholders can use it.
2	Data moves through layers: Raw (as-is) → Staging (cleaned) → Warehouse (trusted).
3	ETL: transform BEFORE loading. Use when data is structured, regulated, stable.
4	ELT: load raw FIRST, transform inside the warehouse using SQL. Use for large-scale, cloud-native pipelines.
5	In both ETL and ELT, we use all 3 files — sales + customers + inventory. The final enriched table has 9 columns.
6	Full Load: reload everything every run. Simple but expensive. Use for small data or first loads.
7	Incremental Load: load only new rows using a watermark. Fast and efficient. Standard in production.
8	DuckDB lets us run production-quality SQL pipelines locally — same concepts as Snowflake or BigQuery.

10 Self-Check Questions

Q1. A retail company has structured sales data and a quarterly compliance audit.

ETL or ELT? Why?

Q2. Your team collects 5 million clickstream events daily. Data science also needs raw data.

ETL or ELT? Why?

Q3. In the ELT code, why LEFT JOIN instead of INNER JOIN?

What rows would an INNER JOIN drop?

Q4. What does the QUALIFY clause do? What problem does it solve?

Q5. Your pipeline uses incremental load with order_date as the watermark.

A vendor batch-updates 3 old orders and sets order_date to today.

Are those orders included in tonight's incremental run? Should they be?

Q6. When would you use Full Load even if Incremental Load is available?

Coming up next in Session 1: DAGs, Orchestration, and Error Handling.

Session 2: Data Quality — "the pipeline ran, but can we trust the data?"

Trainer: Umashankar K H | Programme: FDAIE | Module 2, Session 1